DBI Mitschrift

06.11.2026 – Embedding vs. Referencing, CAP, 3 Vs, Historisierung

Inhaltsverzeichnis

1. Embedding vs. Referencing 1
   1.1. Beispiel: Social-Media-Post (Embedding) 1
   1.2. Lösung: Referencing 1
   1.3. Vergleich 2
2. CAP-Theorem 2
3. Big Data: Die 3 Vs 3
   3.1. Beispiel zu Variety: Verschiedene Datenquellen 3
   3.2. Beispiel: Verschiedene Post-Typen 3
   3.3. Abfragen bei unterschiedlichen Feldern 4
4. Rückblick: Historisierung 4
   4.1. In relationalen Datenbanken 4
   4.2. In JSON / MongoDB 4

1. Embedding vs. Referencing

1.1. Beispiel: Social-Media-Post (Embedding)

Die Kommentare werden direkt im Post-Dokument **eingebettet**:

```
db.socialmediapost.insertOne({
  _id: "BN_061126",
  title: "ich mag sushi",
  text: "…",
  comments: [
    { _id: 1, user: "anna", text: "same" },
    { _id: 2, user: "ben",  text: "lachs > thunfisch" }
  ]
})
```

Vorteil: Post und Kommentare kommen mit **einer** Abfrage, ohne Join.

Problem: Wenn der Post ultra viral geht, wächst das `comments`-Array immer weiter – das **Limit von 16 MB pro Dokument** kann erreicht werden.

1.2. Lösung: Referencing

Deswegen werden die Kommentare in eine **eigene Collection** ausgelagert und verweisen auf den Post:

```
db.comments.insertOne({
  _id: 1,
  post_id: "BN_061126",   // Referenz auf den Post
  user: "anna",
  text: "same"
})
```

Zusammengeführt wird beim Lesen mit einem **Lookup** (`$lookup`) – das entspricht einem Join in SQL.

> **Ergänzung:** Beispiel für einen Lookup. Das Ergebnis ist der Post mit einem Array `comments`, das alle passenden Kommentare enthält:
>
> ```
> db.socialmediapost.aggregate([
>   { $match: { _id: "BN_061126" } },
>   { $lookup: {
> ```

```
      from: "comments",
      localField: "_id",
      foreignField: "post_id",
      as: "comments"
  } }
])
```

Damit der Lookup schnell ist, sollte `post_id` einen Index haben:
`db.comments.createIndex({ post_id: 1 }).`

1.3. Vergleich

	Embedding	**Referencing**
Lesen	eine Abfrage, sehr schnell	braucht `$lookup` (Join)
Größe	begrenzt durch 16 MB	unbegrenzt viele Dokumente
Ändern	Daten evtl. mehrfach gespeichert	nur an einer Stelle
Passt für	1:1, 1:wenige, wird zusammen gelesen	1:viele, n:m, wächst unbegrenzt

Ergänzung: Mischform (*Subset Pattern*): Die neuesten z.B. 3 Kommentare werden im Post eingebettet (für die Vorschau), alle Kommentare liegen zusätzlich in der eigenen Collection.

Prüfungsrelevant:
- Im Gegensatz zur relationalen DB kann man eine **Referenz angeben, die nicht existiert** – es gibt keine Fremdschlüssel-Constraints bzw. referentielle Integrität.
- **Embedding:** Daten werden gemeinsam gelesen und die Menge ist begrenzt.
- **Referencing:** Daten wachsen unbegrenzt (z.B. Kommentare ⟶ 16-MB-Limit) oder werden von mehreren Dokumenten gebraucht.

2. CAP-Theorem

Ein **verteiltes** Datenbanksystem (mehrere Server/Knoten) kann von diesen drei Eigenschaften **höchstens zwei gleichzeitig** garantieren:

C – Consistency Jeder Lesezugriff liefert den zuletzt geschriebenen Wert, egal auf welchem Knoten.
A – Availability Jede Anfrage bekommt eine Antwort, auch wenn Knoten ausgefallen sind.
P – Partition Tolerance Das System funktioniert weiter, auch wenn die Verbindung zwischen Knoten unterbrochen ist.

Da Netzwerkausfälle in verteilten Systemen immer passieren können, muss man **P** praktisch immer haben. Die eigentliche Entscheidung ist also: **CP** oder **AP**.

Typ	**Verhalten bei Netzwerkausfall**	**Beispiele**
CP	lieber **keine** Antwort als eine veraltete	MongoDB (Standard), Banken
AP	antwortet immer, Daten können kurz veraltet sein (*eventual consistency*)	Cassandra, CouchDB, Social Media

Prüfungsrelevant:
- CAP: Consistency, Availability, Partition Tolerance – **max. 2 von 3**.

- In verteilten Systemen ist P Pflicht → Wahl zwischen **CP** und **AP**.

3. Big Data: Die 3 Vs

Volume Wie **groß** sind die Datenbestände? Wenn ein Server nicht mehr reicht, muss **horizontal skaliert** werden (mehr Server, in MongoDB: *Sharding*) statt vertikal (stärkerer Server).

Velocity Wie **schnell** kommen neue Daten dazu und wie schnell müssen sie abgefragt werden können (z.B. Sensordaten, Live-Feeds)?

Variety Viele **verschiedene Datenquellen**, die **unterschiedlich formatierte** Daten liefern (strukturiert, halbstrukturiert wie JSON, unstrukturiert wie Bilder/Text).

3.1. Beispiel zu Variety: Verschiedene Datenquellen

Ein Getränkehändler sammelt Verkaufsdaten aus drei Quellen. Jede Quelle liefert die Daten anders:

```
db.verkaeufe.insertMany([
  // Kassensystem im Geschäft
  { quelle: "kasse", filiale: "Wien 1",
    artikel: "Redbull", menge: 2, preis: 1.49,
    zeit: ISODate("2026-11-06T10:15:00Z") },

  // Online-Shop: mehrere Artikel pro Bestellung, mit Lieferadresse
  { quelle: "webshop", bestellnr: "WS-4711",
    positionen: [
      { artikel: "Redbull", menge: 24 },
      { artikel: "Wasser", menge: 6 }
    ],
    lieferung: { plz: "1010", ort: "Wien" } },

  // Getränkeautomat: nur Automaten-ID und Schacht
  { quelle: "automat", automat_id: "A-17",
    schacht: 3, betrag_cent: 250 }
])
```

In einer relationalen DB müsste man alle Formate zuerst in **ein** festes Schema umwandeln. In MongoDB können alle drei Formate in derselben Collection gespeichert werden.

3.2. Beispiel: Verschiedene Post-Typen

Auch innerhalb einer Anwendung haben Dokumente oft verschiedene Felder:

```
db.socialmediapost.insertMany([
  { _id: "BN_061126", type: "text",
    title: "ich mag sushi", text: "…" },

  { _id: "BN_071126", type: "image",
    title: "mein sushi", imageUrl: "sushi.png",
    width: 1080, height: 1080 },

  { _id: "BN_081126", type: "poll",
    title: "lieblingssushi?",
    options: ["Maki", "Nigiri", "Sashimi"],
    votes: [12, 30, 7] }
])
```

In SQL bräuchte man dafür entweder eine Tabelle mit vielen `NULL`-Spalten oder eine eigene Tabelle pro Post-Typ.

3.3. Abfragen bei unterschiedlichen Feldern

Weil nicht jedes Dokument jedes Feld hat, prüft man mit `$exists`, ob ein Feld vorhanden ist, oder filtert nach einem Typ-Feld:

```
// alle Posts mit Bild
db.socialmediapost.find({ imageUrl: { $exists: true } })

// alle Umfragen
db.socialmediapost.find({ type: "poll" })

// alle Verkäufe aus dem Webshop
db.verkaeufe.find({ quelle: "webshop" })
```

> **Ergänzung:** Ein Feld, das es im Dokument nicht gibt, ist **nicht** dasselbe wie ein Feld mit dem Wert `null`. `find({ imageUrl: null })` findet beides, `find({ imageUrl: { $exists: false } })` nur die Dokumente ohne das Feld.

> **Prüfungsrelevant:**
> - Die 3 Vs: **Volume** (Menge), **Velocity** (Geschwindigkeit), **Variety** (Vielfalt der Quellen und Formate).
> - **Variety** in MongoDB: Dokumente einer Collection können **unterschiedliche Felder und Strukturen** haben (schemalos).
> - Vorteil: flexibel, neue Felder ohne `ALTER TABLE`.
> - Nachteil: Die **Anwendung** muss mit fehlenden Feldern umgehen (z.B. mit `$exists` oder einem Typ-Feld wie `type`/`quelle`).

4. Rückblick: Historisierung

Historisierung heißt, dass alte Werte nicht überschrieben, sondern **aufbewahrt** werden, z.B. um später zu sehen, was ein Produkt an einem bestimmten Tag gekostet hat.

4.1. In relationalen Datenbanken

Neben der eigentlichen Tabelle gibt es eine **extra Historien-Tabelle**. Bei jeder Änderung wird der alte Stand dort mit Gültigkeitszeitraum gespeichert:

```
CREATE TABLE produkt (
  id    INT PRIMARY KEY,
  name  VARCHAR(50),
  preis DECIMAL(6,2)
);

CREATE TABLE produkt_hist (
  id          INT,
  name        VARCHAR(50),
  preis       DECIMAL(6,2),
  gueltig_von DATE,
  gueltig_bis DATE,
  PRIMARY KEY (id, gueltig_von)
);
```

4.2. In JSON / MongoDB

Für jeden Tag wird ein **neues Dokument** angelegt. Die Historisierung steckt direkt **in der ID**: Produktname + Datum.

```
db.preise.insertMany([
  { _id: "Redbull20261010", produkt: "Redbull",
```

```
    datum: ISODate("2026-10-10"), preis: 1.39 },
  { _id: "Redbull20261011", produkt: "Redbull",
    datum: ISODate("2026-10-11"), preis: 1.49 }
])

// Preis an einem bestimmten Tag
db.preise.find({ _id: "Redbull20261011" })

// ganze Historie von Redbull im Oktober 2026
db.preise.find({ _id: /^Redbull202610/ })
```

Ergänzung: Die Suche mit `/^Redbull202610/` funktioniert schnell, weil der Regex mit `^` am Anfang verankert ist und so der Index auf `_id` genutzt werden kann. Das Datum zusätzlich als eigenes Feld `datum` zu speichern macht Abfragen nach Zeiträumen einfacher.

Prüfungsrelevant:

- **Relational:** eigene Historien-Tabelle mit `gueltig_von` / `gueltig_bis`.
- **JSON/MongoDB:** pro Tag ein neues Dokument, Datum steckt in der **ID** (z.B. `Redbull20261011`).